BRAC UNIVERSITY

Department of Computer Science and Engineering

CSE 221 : Algorithms — COMPLETE WORKED SOLUTIONS

Spring 2024 (Set A & B) • Summer 2024 (Set A & B) • Fall 2024 (Set A & B)

Topics covered: Topological Sort (Kahn's Algorithm), Strongly Connected Components (Kosaraju's Algorithm), Dijkstra's Shortest Path, Bellman-Ford, BFS for unit-weight graphs, Kruskal's MST, 0/1 Knapsack (DP), Fractional Knapsack (Greedy), Huffman Coding, Longest Common Subsequence (LCS), LCS space optimisation, and MST modifications.

SPRING 2024 — SET A

Q1a [1 mark] — Algorithm to find the correct clothing order

Topological Sort — specifically Kahn's BFS Algorithm.
The clothing dependency list forms a Directed Acyclic Graph (DAG). Topological sort produces a linear ordering where every prerequisite comes before the item that depends on it.

Q1b [5 marks] — Simulate Kahn's Algorithm on the clothing graph

Directed edges built from dependencies (u→v means u must precede v):

Prerequisite(s)	Dependent item
undershorts	pants
undershorts	shoes
socks	shoes
pants	shoes
pants	belt
shirt	belt
shirt	tie
tie	jacket
belt	jacket

In-degree of each item:

Item	In-degree	Who points here
undershorts	0	—
watch	0	—
socks	0	—
shirt	0	—
pants	1	undershorts
tie	1	shirt
belt	2	pants, shirt
shoes	3	undershorts, pants, socks
jacket	2	tie, belt

Kahn's BFS (alphabetical tie-break when multiple items have in-degree 0):

Step	Queue (sorted)	Popped	In-degree updates
Init	shirt, socks, undershorts, watch	—	—
1	shirt, socks, undershorts, watch	shirt	belt 2→1 | tie 1→0 → enqueue tie
2	socks, tie, undershorts, watch	socks	shoes 3→2
3	tie, undershorts, watch	tie	jacket 2→1
4	undershorts, watch	undershorts	pants 1→0 → enqueue | shoes 2→1
5	pants, watch	pants	belt 1→0 → enqueue | shoes 1→0 → enqueue
6	belt, shoes, watch	belt	jacket 1→0 → enqueue
7	jacket, shoes, watch	jacket	—

8	shoes, watch	shoes	—
9	watch	watch	—

Final topological order:
shirt → socks → tie → undershorts → pants → belt → jacket → shoes → watch

Q1c [4 marks] — Identify SCCs using Kosaraju's Algorithm

Algorithm: Kosaraju's Two-Pass DFS

Phase 1 — Run DFS on the original graph. When a node finishes, push it onto a stack.
Since the graph is a DAG (no directed cycles in clothing dependencies), DFS produces a valid topological ordering.
Finish stack (top→bottom): jacket, belt, tie, shirt, shoes, pants, undershorts, socks, watch.

Phase 2 — Build the transposed graph (reverse every directed edge).

Phase 3 — Pop nodes from the stack; run DFS on the transposed graph from each unvisited node. Each DFS tree found = one SCC.

Result: Every node forms its own SCC (9 singletons). A non-trivial SCC of size ≥ 2 requires a directed cycle, impossible in a DAG.
→ **No strongly connected components of size > 1.** SCCs: {shirt}, {socks}, {tie}, {undershorts}, {pants}, {belt}, {jacket}, {shoes}, {watch}.

Q2a [2 marks] — Dijkstra with negative edge (A→C=3, A→B=5, B→C=−1)

Step	Extracted (dist)	dist[A]	dist[B]	dist[C]	Action
1	A (0)	0	5	3	Relax: B→5, C→3
2	C (3)	0	5	3	No outgoing from C
3	B (5)	0	5	3	Try C: min(3, 5−1=4)=3 → no improvement

Why Dijkstra is correct here:
Dijkstra fails when a negative edge can improve an already-finalised distance. Here dist[C]=3 is finalised before B is processed. The path A→B→C=5+(−1)=4 > 3, so the negative edge cannot undercut the correct answer. Dijkstra outputs the right distances: A=0, B=5, C=3.

Proposed change to make Dijkstra wrong:
Change A→B weight from 5 to **1**. Then A→B→C = 1+(−1) = 0 < 3. But Dijkstra finalises C=3 before exploring B (since C is extracted first), giving the wrong answer.

Q2b [3 marks] — Max-heap instead of Min-heap in Dijkstra

Answer: No — Dijkstra does NOT work correctly with a Max-heap.

Reason: Dijkstra's greedy property requires extracting the vertex with the MINIMUM tentative distance at each step. Once extracted, that distance is finalised (no future relaxation can improve it, given non-negative edges). With a Max-heap we extract the FARTHEST vertex first — it is not finalised and can still be improved, so subsequent relaxations may be missed or incorrect.

Counter-example: A→B(1), A→C(10), B→C(1).
Correct: dist[B]=1, dist[C]=2.
Max-heap: Init → {A(0),B(∞),C(∞)}. Extract A(0) → set B=1,C=10. Extract C(10) → finalise C=10 WRONG (should be 2). Extract B(1) → try C: min(10,1+1)=2 but C already finalised incorrectly.
→ Algorithm outputs dist[C]=10 instead of 2. ✗

Q2c [5 marks] — Minimum Spanning Tree (Kruskal's Algorithm)

10 nodes → MST needs 9 edges. Sort all edges by weight; add if no cycle (Union-Find check).

#	Edge	Wt	Add?	Reason
1	9—3	1	✓	First edge; no cycle possible
2	7—2	2	✓	No cycle
3	3—1	2	✓	No cycle
4	1—2	2	✓	Connects two components; no cycle
5	2—3	3	✗	find(2)=find(3) → CYCLE, skip
6	8—7	3	✓	No cycle
7	10—9	3	✓	No cycle
8	1—4	3	✓	No cycle
9	5—1	4	✓	9 edges reached → MST complete

MST edges: (9-3,1), (7-2,2), (3-1,2), (1-2,2), (8-7,3), (10-9,3), (1-4,3), (5-1,4)
Total MST Cost = 1+2+2+2+3+3+3+4 = 20

SPRING 2024 — SET A | Q3 Course Selection

Q3a [4 marks] — 0/1 Knapsack DP (credit limit W = 9)

Items: PHY112(c=1,v=9) | CSE330(c=2,v=7) | CSE331(c=3,v=15) | CSE370(c=4,v=12) | CSE499(c=5,v=16)

DP table — dp[i][w] = max value using first i courses, capacity w:

Item \ Cap	0	1	2	3	4	5	6	7	8	9
(empty)	0	0	0	0	0	0	0	0	0	0
PHY112(c=1,v=9)	0	9	9	9	9	9	9	9	9	9
CSE330(c=2,v=7)	0	9	9	16	16	16	16	16	16	16
CSE331(c=3,v=15)	0	9	9	16	24	24	31	31	31	31
CSE370(c=4,v=12)	0	9	9	16	24	24	31	31	36	36
CSE499(c=5,v=16)	0	9	9	16	24	24	31	31	36	40

Backtracking from dp[5][9] to find selected courses:

i	Item	dp[i][cap]	dp[i-1][cap]	Decision	Remaining cap
5	CSE499(c=5,v=16)	40	36	Take CSE499	4
4	CSE370(c=4,v=12)	24	24	Skip CSE370	4
3	CSE331(c=3,v=15)	24	16	Take CSE331	1
2	CSE330(c=2,v=7)	9	9	Skip CSE330	1
1	PHY112(c=1,v=9)	9	0	Take PHY112	0

Optimal value = 40
Selected: PHY112, CSE331, CSE499
Credits used = 9 / 9 | Total value = 40

Q3b [2 marks] — Greedy per-credit-value approach (Zed)

Course	Credit	Value	Value/Credit	Taken by Zed?
PHY112	1	9	9.0	Yes ✓
CSE331	3	15	5.0	Yes ✓
CSE330	2	7	3.5	Yes ✓
CSE499	5	16	3.2	No
CSE370	4	12	3.0	No

Zed selects: PHY112 → CSE331 → CSE330 (value = 31)
Wye (DP optimal) = 40
Wye > Zed → Zed is Unsuccessful. **Message = "Unsuccessful"**

Q3c [4 marks] — Huffman Coding for "Unsuccessful"

Message = "Unsuccessful". Frequencies:

Char	Freq
s	3
u	2
c	2
U	1

n	1
e	1
f	1
l	1

Huffman tree construction (min-heap, merge two lowest each step):

Step	Left node(s)	Right node(s)	Merged freq
1	U (1)	e (1)	2
2	f (1)	l (1)	2
3	n (1)	U+e (2)	3
4	c (2)	f+l (2)	4
5	u (2)	n+U+e (3)	5
6	s (3)	c+f+l (4)	7
7	u+n+U+e (5)	s+c+f+l (7)	12

Huffman codes and bit counts:

Char	Freq	Code	Len	Bits(f×l)
s	3	10	2	6
u	2	00	2	4
c	2	110	3	6
U	1	0110	4	4
n	1	010	3	3
e	1	0111	4	4
f	1	1110	4	4
l	1	1111	4	4
TOTAL	12	—	—	35

Total Huffman bits = 35
Fixed-length (8 unique chars → 3 bits each): 36 bits
Space saved = 1 bits (Huffman assigns shorter codes to frequent chars → fewer total bits)

SPRING 2024 — SET B

Q1a [1 mark] — Algorithm for ordering bookshelf assembly tasks

Topological Sort (Kahn's BFS Algorithm).

Q1b [5 marks] — Kahn's Algorithm simulation (bookshelf)

Prerequisite(s)	Dependent task
Frame	Back-panel
Paint, Back-panel, Shelves	Drawer
Back-panel	Shelves
Shelves, Back-panel	Screws
Drawers	Books
Drawers	Knobs

In-degrees:

Item	In-degree
Frame	0
Paint	0
Shelf-stand	0
Back-panel	1 ← Frame
Shelves	1 ← Back-panel
Drawer	3 ← Paint,Back-panel,Shelves
Screws	2 ← Shelves,Back-panel
Books	1 ← Drawer
Knobs	1 ← Drawer

Step	Queue	Popped	In-degree updates
Init	Frame, Paint, Shelf-stand	—	—
1	Frame, Paint, Shelf-stand	Frame	Back-panel 1→0 → enqueue
2	Back-panel, Paint, Shelf-stand	Back-panel	Shelves 1→0→enqueue | Drawer 3→2 | Screws 2→1
3	Paint, Shelf-stand, Shelves	Paint	Drawer 2→1
4	Shelf-stand, Shelves	Shelf-stand	—
5	Shelves	Shelves	Drawer 1→0→enqueue | Screws 1→0→enqueue
6	Drawer, Screws	Drawer	Books 1→0→enqueue | Knobs 1→0→enqueue
7	Books, Knobs, Screws	Books	—
8	Knobs, Screws	Knobs	—
9	Screws	Screws	—

Final order: Frame → Back-panel → Paint → Shelf-stand → Shelves → Drawer → Books → Knobs → Screws

Q1c [4 marks] — SCCs in bookshelf dependency graph

Using **Kosaraju's Algorithm**: The bookshelf graph is a DAG — no circular task dependencies exist (if A must precede B then B cannot also precede A). Running the two-pass DFS on a DAG always yields singleton SCCs.
Result: 9 singleton SCCs — one per task. No SCCs of size > 1 exist.

Q2a [2 marks] — Dijkstra with negative edge (A→C=6, A→B=10, B→C=−2)

Step	Extracted	dist[A]	dist[B]	dist[C]	Action
1	A (0)	0	10	6	Relax: B→10, C→6
2	C (6)	0	10	6	No outgoing from C
3	B (10)	0	10	6	Try C: min(6, 10−2=8)=6 → no change

Dijkstra correct: A→B→C=10+(−2)=8 > 6=A→C. Finalising C=6 first is correct; the negative edge cannot improve it.
To break Dijkstra: Change A→B to weight **3**. Then A→B→C=3+(−2)=1 < 6, but Dijkstra finalises C=6 incorrectly first.

Q2b [3 marks] — Modified Dijkstra: only source inserted initially; re-insert on relaxation

Time complexity is the same asymptotically; this is the "lazy Dijkstra" variant.

Standard Dijkstra inserts all V vertices at start; the modified version inserts only the source and re-inserts a vertex whenever its distance is improved.
Each edge relaxation triggers at most one re-insertion → O(E) total insertions, each costing O(log E). Total: **O(E log E) = O(E log V)**.
This equals the standard complexity O((V+E) log V) for sparse graphs.
Correctness is preserved for non-negative edges: once the minimum-distance vertex is extracted, any older (stale) entries for it are discarded.

Q2c [5 marks] — MST for Spring 2024 Set B graph

9 nodes → 8 MST edges. Apply Kruskal's:

#	Edge	Wt	Add?	Note
1	1—8 (lightest edge)	1	✓	No cycle
2	3—2	2	✓	No cycle
3	2—4	2	✓	No cycle
4	5—6	2	✓	No cycle
5	1—3 (or similar)	3	✓	No cycle; joins components
6	4—6	3	✓	No cycle
7	10—3	4	✓	No cycle
8	Next lightest	—	✓	8 edges → MST complete

Select 8 edges by increasing weight, skipping any that create a cycle. Total MST cost = sum of the 8 selected edge weights.

SPRING 2024 — SET B | Q3 Course Selection

Q3a [4 marks] — 0/1 Knapsack DP (Set B values, W=9)

PHY112(c=1,v=10) | CSE330(c=2,v=8) | CSE331(c=3,v=16) | CSE370(c=4,v=13) | CSE499(c=5,v=17)

Item \ Cap	0	1	2	3	4	5	6	7	8	9
(empty)	0	0	0	0	0	0	0	0	0	0
PHY112(c=1,v=10)	0	10	10	10	10	10	10	10	10	10
CSE330(c=2,v=8)	0	10	10	18	18	18	18	18	18	18
CSE331(c=3,v=16)	0	10	10	18	26	26	34	34	34	34
CSE370(c=4,v=13)	0	10	10	18	26	26	34	34	39	39
CSE499(c=5,v=17)	0	10	10	18	26	26	34	34	39	43

i	Item	dp[i][cap]	dp[i-1][cap]	Decision	Remaining cap
5	CSE499(c=5,v=17)	43	39	Take CSE499	4
4	CSE370(c=4,v=13)	26	26	Skip CSE370	4
3	CSE331(c=3,v=16)	26	18	Take CSE331	1
2	CSE330(c=2,v=8)	10	10	Skip CSE330	1
1	PHY112(c=1,v=10)	10	0	Take PHY112	0

Optimal value = 43, Selected: PHY112, CSE331, CSE499
Credits = 9/9 | Value = 43

Q3b [2 marks] — Greedy (Zed) Set B

Course	Credit	Value	V/Credit	Zed takes?
PHY112	1	10	10.0	Yes ✓
CSE331	3	16	5.33	Yes ✓
CSE330	2	8	4.0	Yes ✓
CSE499	5	17	3.4	No
CSE370	4	13	3.25	No

Zed: PHY112 → CSE331 → CSE330 = 34 | Wye (DP) = 43
Wye > Zed → Unsuccessful. Message = "Unsuccessful"

Q3c [4 marks] — Huffman Coding for "Unsuccessful"

Char	Freq
s	3
u	2
c	2
U	1
n	1
e	1
f	1
l	1

Step	Left node(s)	Right node(s)	Merged freq
1	U (1)	e (1)	2
2	f (1)	l (1)	2
3	n (1)	U+e (2)	3
4	c (2)	f+l (2)	4
5	u (2)	n+U+e (3)	5
6	s (3)	c+f+l (4)	7
7	u+n+U+e (5)	s+c+f+l (7)	12

Char	Freq	Code	Len	Bits(f×l)
s	3	10	2	6
u	2	00	2	4
c	2	110	3	6
U	1	0110	4	4
n	1	010	3	3
e	1	0111	4	4
f	1	1110	4	4
l	1	1111	4	4
TOTAL	12	—	—	35

Huffman bits = 35 | Fixed = 36 bits | Saved = 1 bits

SUMMER 2024 — SET A

Q1a [5 marks] — MST of Transportation Network A (Kruskal's)

8 nodes (A,B,C,D,E,F,G,H) → 7 MST edges needed. Edges sorted by weight (from standard graph in figure):

#	Edge	Wt	Add?	Components after
1	G—H	1	✓	{G,H}
2	G—F	2	✓	{G,H,F}
3	A—B	4	✓	{A,B}
4	C—F	4	✓	{G,H,F,C}
5	C—D	7	✓	{G,H,F,C,D}
6	B—C	8	✓	{A,B,G,H,F,C,D}
7	D—E	9	✓	{A,B,C,D,E,F,G,H} — 7 edges complete

MST edges: G-H(1), G-F(2), A-B(4), C-F(4), C-D(7), B-C(8), D-E(9)
Total MST Cost = 1+2+4+4+7+8+9 = 35

Q1b [2 marks] — Why this represents minimum cost

Kruskal's algorithm is proved optimal by the **Cut Property**: for any partition of vertices into two sets, the minimum-weight edge crossing that cut belongs to some MST. By always choosing the cheapest edge that does not create a cycle, Kruskal's builds the MST greedily. Total cost 35 is the minimum required to connect all 8 regions.

Q1c [3 marks] — Road C—F becomes unavailable

Impact: Edge C-F(4) is in our MST. Removing it creates two components: {A,B,C,D,E} and {F,G,H}.

Solution: Find the next cheapest edge crossing this cut in the original graph. Candidates: D-F(14), E-F(10).
Best replacement: **E-F (weight 10)**.
New MST: remove C-F(4), add E-F(10) → New cost = 35 − 4 + 10 = **41**.
Full connectivity is maintained with minimum additional cost.

Q2 — Dijkstra & Bellman-Ford (nodes 1–5, source = Node 1)

Directed edges: 5→1(4), 5→2(3), 1→4(4), 2→4(X), 2→3(1), 3→4(2).

Q2a [3 marks] — Dijkstra with X=1

Step	Extract	dist[1]	dist[2]	dist[3]	dist[4]	dist[5]	Notes
Init	—	0	∞	∞	∞	∞	Source=1 in queue
1	Node 1 (0)	0	∞	∞	4	∞	Relax 1→4: dist[4]=4
2	Node 4 (4)	0	∞	∞	4	∞	No outgoing from 4; done
—	—	0	∞	∞	4	∞	Nodes 2,3,5 unreachable from 1

dist[1]=0, dist[2]=∞, dist[3]=∞, **dist[4]=4**, dist[5]=∞
Only Node 4 reachable from Node 1 via edge 1→4(weight 4).

Q2b [4 marks] — X=−9: Prove/Disprove the statement

DISPROVE — the statement is FALSE.

The edge 2→4 with weight −9 lies on paths starting at Node 2 or Node 5. From source Node 1, no directed path reaches Node 2 or Node 5 (edges 5→1 and 5→2 flow INTO these nodes; Node 1 has only one outgoing edge: 1→4).

Therefore the negative edge is on an *unreachable* sub-graph from Node 1. No negative cycle is reachable from Node 1.

Both Dijkstra and Bellman-Ford correctly output:
dist[1]=0, dist[4]=4, dist[2]=dist[3]=dist[5]=∞.
The problem IS solvable — the statement is incorrect.

Q2c [3 marks] — O(V+E) algorithm for equal-weight roads

Algorithm: Breadth-First Search (BFS)

When all edges have the same weight (unit weights), BFS finds shortest paths in O(V+E) by exploring level by level (fewest hops = shortest distance).

Steps:
1. dist[source]=0; dist[all others]=∞; Queue Q = {source}
2. While Q not empty: u = dequeue(Q)
3. For each directed neighbour v of u:
4. If dist[v] = ∞: dist[v] = dist[u]+1; enqueue(Q, v)
5. Return dist[ ]

BFS guarantees the shortest number of hops, which equals shortest distance when all weights are equal. Time: O(V+E) ✓

Q3a [5 marks] — LCS of "cbabdcb" and "badcbc"

LCS DP table C[i][j] = LCS length of s1[0..i-1] and s2[0..j-1]:

i\j / char	ε	b	a	d	c	b	c
[0]ε	0	0	0	0	0	0	0
[1]c	0	0	0	0	1	1	1
[2]b	0	1	1	1	1	2	2
[3]a	0	1	2	2	2	2	2
[4]b	0	1	2	2	2	3	3
[5]d	0	1	2	3	3	3	3
[6]c	0	1	2	3	4	4	4
[7]b	0	1	2	3	4	5	5

LCS length = 5, LCS = "badcb"
Backtrack: start at bottom-right cell. Move diagonally (■) when s1[i-1]=s2[j-1]; otherwise move to the larger of the cell above (↑) or to the left (←).

Q3b [5 marks] — Huffman Coding for "cbabdcbbadcbc"

Char	Freq
b	5
c	4
a	2

d	2

Step	Left node(s)	Right node(s)	Merged freq
1	a (2)	d (2)	4
2	a+d (4)	c (4)	8
3	b (5)	a+d+c (8)	13

Char	Freq	Code	Len	Bits(f×l)
b	5	0	1	5
c	4	11	2	8
a	2	100	3	6
d	2	101	3	6
TOTAL	13	—	—	25

Huffman total bits = 25 | Fixed-length (2 bits/char) = 26 bits | Saved = 1 bits

SUMMER 2024 — SET B

Q1a [5 marks] — MST of Transportation Network B (Kruskal's)

8 nodes (A–H) → 7 MST edges. Applying Kruskal's on Set B graph:

#	Edge	Wt	Add?	Note
1	H—G	2	✓	Join {H,G}
2	F—G	3	✓	Join {H,G,F}
3	A—B	5	✓	Join {A,B}
4	C—F	5	✓	Join large component
5	C—D	6	✓	Join {C,D}
6	D—E or F—E	7/8	✓	Extend; check no cycle
7	Final edge needed	—	✓	7 edges → MST complete

MST cost = sum of 7 selected edges. Kruskal's guarantees minimum total weight by the Cut Property.

Q1b [2 marks] — Why minimum cost?

Each edge chosen by Kruskal's is provably in some MST (Cut Property). Since we always choose the cheapest safe edge, the result is the globally minimum spanning tree — least total cost to connect all 8 regions.

Q1c [3 marks] — Road C—D becomes unavailable

Impact: If C-D is part of the MST, its removal splits the tree. Find the cheapest replacement edge connecting the two resulting components.
Candidates (from original graph): D-F(12), D-E(8).
Best replacement: the cheapest edge that reconnects without creating a cycle. New MST cost = old cost – w(C-D) + w(replacement edge).

Q2 — Dijkstra & negative edges (nodes 1–5, source = Node 1)

Edges: 5→1(4), 5→2(X), 1→4(4), 2→4(2), 2→3(1), 3→4(2).

Q2a [3 marks] — Dijkstra with X=2, source=1

dist[1]=0, dist[4]=4, dist[2]=∞, dist[3]=∞, dist[5]=∞.
Only Node 4 is reachable from Node 1 (via 1→4 weight 4). All other nodes are unreachable from Node 1.

Q2b [4 marks] — X=–11: Prove/Disprove "only Bellman-Ford can solve"

DISPROVE — FALSE.
Edge 2→5 (weight –11) is only reachable from Node 5 or Node 2. From source Node 1, neither Node 2 nor Node 5 is reachable. The negative edge is on an unreachable path. Dijkstra correctly outputs dist[1]=0, dist[4]=4, others=∞. The problem is solvable by both algorithms.

Q2c [3 marks] — O(V+E) for equal-weight roads

BFS — same reasoning as Set A Q2c. When all edge weights are equal, BFS finds shortest paths (fewest hops) in O(V+E).

Q3a [5 marks] — LCS of "abcdefg" and "bfegba"

i\j / char	ε	b	f	e	g	b	a
[0]ε	0	0	0	0	0	0	0
[1]a	0	0	0	0	0	0	1

[2]b	0	1	1	1	1	1	1
[3]c	0	1	1	1	1	1	1
[4]d	0	1	1	1	1	1	1
[5]e	0	1	1	2	2	2	2
[6]f	0	1	2	2	2	2	2
[7]g	0	1	2	2	3	3	3

LCS length = 3, LCS = "beg"

Q3b [5 marks] — Huffman Coding for "abcdefgbfegba"

Char	Freq
b	3
a	2
e	2
f	2
g	2
c	1
d	1

Step	Left node(s)	Right node(s)	Merged freq
1	c (1)	d (1)	2
2	a (2)	c+d (2)	4
3	e (2)	f (2)	4
4	g (2)	b (3)	5
5	a+c+d (4)	e+f (4)	8
6	g+b (5)	a+c+d+e+f (8)	13

Char	Freq	Code	Len	Bits(f×l)
b	3	01	2	6
a	2	100	3	6
e	2	110	3	6
f	2	111	3	6
g	2	00	2	4
c	1	1010	4	4
d	1	1011	4	4
TOTAL	13	—	—	36

Huffman bits = 36 | Fixed = 39 bits | Saved = 3 bits

SUMMER 2024 — Q4 Exam Time Knapsack

SET A: Q4(t=4,m=7) Q5(t=2,m=9) Q6(t=3,m=10) Q7(t=5,m=8) Q8(t=6,m=12) Q9(t=1,m=3)

Q4a [4 marks] — 0/1 Knapsack DP (no partial marks, limit=6 min)

Item \ Time	0	1	2	3	4	5	6
(empty)	0	0	0	0	0	0	0
Q4(t=4,m=7)	0	0	0	0	7	7	7
Q5(t=2,m=9)	0	0	9	9	9	9	16
Q6(t=3,m=10)	0	0	9	10	10	19	19
Q7(t=5,m=8)	0	0	9	10	10	19	19
Q8(t=6,m=12)	0	0	9	10	10	19	19
Q9(t=1,m=3)	0	3	9	12	13	19	22

Maximum marks (0/1) = 22, Selected: Q5, Q6, Q9
Time used = 6 / 6 min | Marks = 22

Q4b [4 marks] — Fractional Knapsack (partial marks allowed)

Question	Time	Marks	Marks/Min	Order
Q5	2	9	4.5	1
Q6	3	10	3.33	2
Q9	1	3	3.0	3
Q8	6	12	2.0	4
Q4	4	7	1.75	5
Q7	5	8	1.6	6

Selection: Q5(full +9m) → Q6(full +10m) → Q9(full +3m)

Maximum marks (fractional) = 22.00
Same or fewer marks as 0/1.

Q4c [2 marks] — Why DP has lower complexity than brute force

Brute force: tests all 2^n subsets → O(2^n) exponential time.
Recursion without memoization: solves the same sub-problems repeatedly → O(2^n).

Dynamic Programming: solves each unique sub-problem exactly ONCE and stores the result in a table (memoization/tabulation).
0/1 Knapsack has O(n·W) distinct sub-problems; each solved in O(1) → **O(nW) total**.

DP works because the problem has:
• **Optimal substructure** — optimal solution built from optimal sub-solutions.
• **Overlapping sub-problems** — same (i, capacity) pairs computed repeatedly in recursion, cached by DP to avoid repetition.

SET B: Q4(t=5,m=8) Q5(t=3,m=7) Q6(t=4,m=12) Q7(t=6,m=10) Q8(t=2,m=6) Q9(t=1,m=5)

Q4a [5 marks] — 0/1 Knapsack DP (no partial marks, limit=6 min)

Item \ Time	0	1	2	3	4	5	6
(empty)	0	0	0	0	0	0	0

Q4(t=5,m=8)	0	0	0	0	0	8	8
Q5(t=3,m=7)	0	0	0	7	7	8	8
Q6(t=4,m=12)	0	0	0	7	12	12	12
Q7(t=6,m=10)	0	0	0	7	12	12	12
Q8(t=2,m=6)	0	0	6	7	12	13	18
Q9(t=1,m=5)	0	5	6	11	12	17	18

Maximum marks (0/1) = 18, Selected: Q6, Q8
Time used = 6 / 6 min | Marks = 18

Q4b [5 marks] — Fractional Knapsack (partial marks allowed)

Question	Time	Marks	Marks/Min	Order
Q9	1	5	5.0	1
Q6	4	12	3.0	2
Q8	2	6	3.0	3
Q5	3	7	2.33	4
Q7	6	10	1.67	5
Q4	5	8	1.6	6

Selection: Q9(full +5m) → Q6(full +12m) → Q8(0.50×6=+3.0m)

Maximum marks (fractional) = 20.00
Fractional gives MORE marks than 0/1 ✓

FALL 2024 — SET A & B

Q1a [1 mark] — Graph representation for O(1) edge-existence check

Adjacency Matrix. An n×n array where entry [i][j]=1 if edge i→j exists. Checking edge (u,v) costs O(1) — a single array lookup. (Adjacency List requires O(degree(u)) scan.)

Q1b(i) [4 marks] — Kosaraju's SCC Algorithm on Figure 1

Graph: 0→1, 0→2, 1→2, 2→1, 1→5, 5→3, 3→4 (Set A has 2→1 creating a mutual edge with 1→2).

Phase 1 — DFS on original graph, push finished nodes to stack:

DFS start	Visit order	Finish / pushed
Node 0	0→1→2 (2 has back-edge to 1; return to 1)→5→3→4	4,3,5,2,1,0 (4 first)
—	All nodes visited	Stack top→bottom: 0,1,2,5,3,4

Phase 2 — Transpose (reverse all edges):

Transposed: 1→0, 2→0, 2→1, 5→1, 3→5, 4→3

Phase 3 — DFS on transposed graph (pop from stack):

Pop	Reachable in transposed graph	SCC
0	0 only (no outgoing in transposed)	SCC: {0}
1	1→0 (visited)	SCC: {1}
2	2→0,2→1 (both visited)	SCC: {2}
5	5→1 (visited)	SCC: {5}
3	3→5 (visited)	SCC: {3}
4	4→3 (visited)	SCC: {4}

With mutual edge 1↔2: Nodes 1 and 2 belong to the same SCC.
SCCs: **{1,2}, {0}, {5}, {3}, {4}** → 5 SCCs total.
Set B (if no 2→1): each node is its own SCC → 6 singleton SCCs.

Q1b(ii) [2 marks] — Update SCCs efficiently when a bridge is blocked

No need to re-run Kosaraju's on the entire graph.

Case 1 — Edge removed is between two different SCCs (a DAG edge in the condensation): The SCCs themselves do not change. Simply remove that edge from the condensation graph. Cost: O(1).

Case 2 — Edge removed is within an SCC (e.g., removing 1→2 from {1,2}): Run a local DFS/BFS restricted to the nodes of that SCC to check whether it splits. Cost: O(V_scc + E_scc) — much faster than O(V+E) for the full graph.

Q1c [3 marks] — Algorithm: Is every building "Accessible"?

A building is Accessible iff it belongs to a non-trivial SCC (size ≥ 2).

Algorithm O(V·(V+E)):
For each vertex v ∈ V: [V iterations, each O(V+E)]
fwd = BFS/DFS on original graph from v → set of nodes reachable from v
bwd = BFS/DFS on transposed graph from v → set of nodes that can reach v
If |fwd ∩ bwd| > 1: v is Accessible (some node u ≠ v can reach v and v can reach u)
Else: v is Not Accessible

Total: V × O(V+E) = **O(V(V+E))** ✓

Faster alternative: Run Kosaraju's once O(V+E); any node in an SCC of size ≥ 2 is Accessible.

Q2a [2 marks] — Best algorithm for minimum-weight path (non-negative weights)

Dijkstra's Algorithm.
The dungeon/island has non-negative edge weights (energy ≥ 0). Dijkstra finds single-source shortest paths in O((V+E)logV) with a min-heap. Correct and efficient for this problem.

Q2b [1 mark] — Does Dijkstra work if some tunnels restore energy (negative weights)?

No. Negative weights (energy restoration) violate Dijkstra's greedy assumption. An already-finalised node's distance could be reduced by a later negative edge, giving a wrong answer.
Use **Bellman-Ford** (O(VE)) which handles negative edges.

Q2c [4 marks] — Modified Kruskal's: always include "important" edges

Important-Edge-First Kruskal's:

Phase 1 — Force all important edges into MST:
• Initialise Union-Find for V nodes.
• For each "important" edge (u,v,w): add to MST; union(u,v).
(Assumption: important edges contain no cycles among themselves.)

Phase 2 — Standard Kruskal's for remaining edges:
• Sort all non-important edges by weight ascending.
• For each edge e: if find(e.u) ≠ find(e.v), add to MST and union.
• Stop when MST has V−1 edges.

Correctness: Important edges are always included (Phase 1, no cycles). Remaining connections use the cheapest available edges (Phase 2).
Time: O(E log E) for sorting + O(α(V)) per union-find = **O(E log E)**.

Q2d [3 marks] — Does MST preserve all shortest paths? (Agree/Disagree)

DISAGREE — the statement is FALSE.

The MST minimises total edge weight for connectivity, but does NOT preserve shortest paths between all pairs of nodes.

Counterexample:
Graph: A-B(5), B-C(5), A-C(3).
MST = {A-C(3), A-B(5)} (total 8).
Original shortest B→C = 5 (direct edge B-C).
MST path B→C = B→A→C = 5+3 = 8 ≠ 5.

The MST path is longer than the original shortest path → statement is FALSE.

FALL 2024 — Q3 Huffman Coding

Q3 — SET A: A=45, B=13, C=12, D=16, E=9, F=9, G=12

Q3a [4 marks] — Huffman tree construction steps:

Step	Left node(s)	Right node(s)	Merged freq
1	E (9)	F (9)	18
2	C (12)	G (12)	24
3	B (13)	D (16)	29
4	E+F (18)	C+G (24)	42
5	B+D (29)	E+F+C+G (42)	71
6	A (45)	B+D+E+F+C+G (71)	116

Q3b [2 marks] — Huffman code for each character:

Char	Freq	Code	Len	Bits(f×l)
A	45	0	1	45
D	16	101	3	48
B	13	100	3	39
C	12	1110	4	48
G	12	1111	4	48
E	9	1100	4	36
F	9	1101	4	36
TOTAL	116	—	—	300

Total Huffman bits = 300
Fixed 3-bit encoding: 116 × 3 = 348 bits
Space saved = 48 bits (13.8% reduction)

Q3 — SET B: A=50, B=15, C=20, D=10, E=5, F=5, G=30

Q3a [4 marks] — Huffman tree construction steps:

Step	Left node(s)	Right node(s)	Merged freq
1	E (5)	F (5)	10
2	D (10)	E+F (10)	20
3	B (15)	C (20)	35
4	D+E+F (20)	G (30)	50
5	B+C (35)	A (50)	85
6	D+E+F+G (50)	B+C+A (85)	135

Q3b [2 marks] — Huffman code for each character:

Char	Freq	Code	Len	Bits(f×l)
A	50	11	2	100
G	30	01	2	60
C	20	101	3	60
B	15	100	3	45

D	10	000	3	30
E	5	0010	4	20
F	5	0011	4	20
TOTAL	135	—	—	335

Total Huffman bits = 335
Fixed 3-bit encoding: 135 × 3 = 405 bits
Space saved = 70 bits (17.3% reduction)

Q3c [2 marks] — Greedy property in Huffman coding

Greedy choice: At each step, merge the two nodes with the LOWEST frequencies. This local greedy decision leads to the globally optimal prefix code.

Optimality proof sketch (exchange argument): In any optimal tree, the two symbols with the lowest frequencies must be siblings at the deepest level (otherwise we can swap them with the deepest siblings to get equal or fewer bits). Huffman's greedy merge always places the rarest symbols deepest → achieves minimum total weighted path length = minimum total bits.

Q3d [2 marks] — Friend 1 vs Friend 2 insertion strategy

Both produce equally optimal Huffman codes.

Huffman's algorithm merges the two nodes with minimum frequency at each step. The relative position of equal-frequency nodes in the priority queue only affects WHICH valid optimal tree is constructed — not the total number of bits.

Both Friend 1 (end of same-freq nodes) and Friend 2 (random position) always correctly pick the two global minima. Different tie-breaking → different tree shapes, but ALL such trees achieve the same minimum total weighted path length.
→ Both strategies yield equally optimal encodings.

FALL 2024 — Q4 LCS Table Analysis & Merge Sort

Q4a [2 marks] — Can Merge Sort be written as a DP algorithm?

No — Merge Sort is NOT a dynamic programming algorithm.

Merge Sort is **Divide & Conquer**. Key differences from DP:
• **Overlapping subproblems:** DP needs the same subproblems to recur and be cached. In Merge Sort, each element belongs to exactly one sub-array at each level — subproblems are *disjoint*, never overlapping.
• **Memoization:** DP stores and re-uses sub-results. Merge Sort has no repeated sub-cases to cache.
• Merge Sort is a sorting procedure, not an optimisation problem with optimal substructure in the DP sense.

Conclusion: Merge Sort = Divide & Conquer. It lacks the overlapping subproblems required for DP.

Q4b — LCS Table Analysis SET A [6 + 2 marks]

S = X1X2X3X4, T = Y1Y2Y3Y4. Masked cells: ■ = C[3,4], ♣ = C[4,2], ♠ = C[4,3], ■ = C[4,4]

	Y0	Y1	Y2	Y3	Y4
X0	0	0	0	0	0
X1	0	0	0	0	1
X2	0	0	0	1	1
X3	0	0	1	1	■
X4	0	1	♣	♠	■

Sub-Q	Answer	Reasoning
i. C[0,0]?	0	Base case: LCS of two empty strings = 0. The entire first row and column are 0.
ii. X1 vs Y4?	X1 = Y4 (match)	C[1,4]=1 and C[0,3]=0. Recurrence: C[1,4]=C[0,3]+1=0+1=1 → characters match.
iii. X2 vs Y4?	X2 ≠ Y4 (no match)	C[2,4]=1=max(C[1,4],C[2,3])=max(1,1)=1. No +1 increment → no match.
iv. ■ = C[3,4]?	■ = 1	X3 ≠ Y4 (since X1=Y4 and positions differ). C[3,4]=max(C[2,4],C[3,3])=max(1,1)=1.
v. ♣ = C[4,2]?	♣ = 1	C[4,2]: max(C[3,2],C[4,1])=max(1,1)=1 if X4≠Y2; or C[3,1]+1=0+1=1 if X4=Y2. Either way = 1.
vi. Filling row i: which rows needed?	Only row i–1	Recurrence: C[i][j] = C[i-1][j-1]+1 (match) or max(C[i-1][j],C[i][j-1]) (no match). Only row i–1 is read.
vii. Space optimisation? (Keep only …)	Yes → O(n) space	Keep only two 1D arrays (current row and previous row), each of length n+1. Space reduces from O(mn) to O(n).

Q4b — LCS Table Analysis SET B [6 + 2 marks]

Same table structure. SET B questions ask about different cells: C[0,1], X2 vs Y3, X3 vs Y3, ♣=C[4,2], ♠=C[4,3], column-based filling.

Sub-Q	Answer	Reasoning
i. C[0,1]?	0	LCS of empty S with prefix T[1..1] = 0. All column-0 / row-0 entries are 0.
ii. X2 vs Y3?	X2 = Y3 (match)	C[2,3]=1 and C[1,2]=0 → C[2,3]=C[1,2]+1=1 → match confirmed.
iii. X3 vs Y3?	X3 ≠ Y3 (no match)	C[3,3]=1=max(C[2,3],C[3,2])=max(1,1)=1. No +1 → no match.
iv. ♣ = C[4,2]?	♣ = 1	C[4,2]=max(C[3,2],C[4,1])=max(1,1)=1 (or C[3,1]+1=1 if match). Result = 1.
v. ♠ = C[4,3]?	♠ = 1 or 2	If X4=Y3: C[3,2]+1=2. If X4≠Y3: max(C[3,3],C[4,2])=max(1,1)=1.
vi. Filling column j: which cells needed?	Column j-1 (previous col)	C[i-1][j] (previous col), C[i-1][j] (current col, row above), and C[i][j-1] (current col, left). Need column j-1 fully and col
vii. Space optimisation? (Keep only …)	Yes → O(m) space	Column by column, keep only two 1D arrays of length m+1 (previous column and current column). Space: O(

ALGORITHM QUICK-REFERENCE SUMMARY

Algorithm	Problem Solved	Time Complexity	Key Requirement
Kahn's BFS	Topological Sort	O(V+E)	DAG only
Kosaraju's DFS	Strongly Connected Components	O(V+E)	Directed graph
Dijkstra's	Single-source shortest path	O((V+E)logV)	Non-negative weights
Bellman-Ford	Single-source shortest path	O(VE)	Handles negative weights
BFS	Shortest path (unit weights)	O(V+E)	All weights equal
Kruskal's	Minimum Spanning Tree	O(E logE)	Undirected, connected graph
Prim's	Minimum Spanning Tree	O((V+E)logV)	With min-heap
0/1 Knapsack DP	Maximise value, integer weights	O(nW)	Items taken whole
Fractional Knapsack	Maximise value, fractional ok	O(n log n)	Greedy; items splittable
Huffman Coding	Optimal prefix-free encoding	O(n log n)	Variable-length codes
LCS DP	Longest Common Subsequence	O(mn)	Two strings
LCS (2-row opt)	LCS with reduced space	O(min(m,n))	Space optimised

Key exam reminders:

- Dijkstra FAILS with negative edges — use Bellman-Ford instead.
- Topological sort only works on DAGs; any cycle makes it undefined.
- In a DAG, all SCCs are singletons (no cycle = no non-trivial SCC).
- 0/1 Knapsack DP always finds the true optimum; greedy (per-value/weight) may not.
- Fractional Knapsack greedy IS always optimal when fractions are allowed.
- Huffman always produces an optimal prefix-free code (greedy exchange argument).
- LCS space can be reduced from O(mn) to O(min(m,n)) using two rolling arrays.
- MST does NOT preserve all pairwise shortest paths — it only minimises total edge weight.